

Function name	Purpose	Input Types	Example usage	Result
cbrt	Cube root	Numeric	<code>sqrt(lit(8))</code>	2
ceil	Ceiling	Numeric	<code>ceil(lit(8.5))</code>	9
cos	Cosine	Numeric	<code>cos(lit(0.5))</code>	0.877... ^a
sin	Sine	Numeric	<code>sin(lit(0.5))</code>	0.479... ^a
tan	Tangent	Numeric	<code>tan(lit(0.5))</code>	0.546... ^a
exp	Exponent	Numeric	<code>exp(lit(1.0))</code>	2.718... ^a
floor	Ceiling	Numeric	<code>floor(lit(8.5))</code>	8
least	Minimum value	Numerics	<code>least(lit(1), lit(-10))</code>	-10

^a Truncated for display purposes.

Table 3-7. Functions for use on Spark SQL arrays

Function name	Purpose	Example usage	Result
<code>array_contains</code>	If an array contains a value	<code>array_contains(lit(Array(2,3,-1)), 3)</code>	true
<code>sort_array</code>	Sort an array (ascending default)	<code>sort_array(lit(Array(2,3,-1)))</code>	Array(-1,2,3)
<code>explode</code>	Create a row for each element in the array - often useful when working with nested JSON records. Either takes a column name or additional function mapping from row to iterator of case classes.	<code>explode(lit(Array(2,3,-1)), "murh")</code>	Row(2), Row(3), Row(-1)

Beyond simply filtering out data, you can also produce a DataFrame with new columns or updated values in old columns. Spark uses the same expression syntax we discussed for filter, except instead of having to include a condition (like testing for equality), the results are used as values in the new DataFrame. To see how you can use select on complex and regular data types, [Example 3-15](#) uses the Spark SQL explode function to turn an input DataFrame of PandaPlaces into a DataFrame of

just `PandaInfo` as well as computing the “squishness” to “hardness” ratio of each panda.

Example 3-15. Spark SQL select and explode operators

```
val pandaInfo = pandaPlace.explode(pandaPlace("pandas")){
  case Row(pandas: Seq[Row]) =>
    pandas.map{
      case Row(id: Long, zip: String, pt: String, happy: Boolean, attrs: Seq[Double]) =>
        RawPanda(id, zip, pt, happy, attrs.toArray)
    }
}
pandaInfo.select(
  (pandaInfo("attributes")(0) / pandaInfo("attributes")(1))
  .as("squishyness"))
```



When you construct a sequence of operations, the generated column names can quickly become unwieldy, so the `as` or `alias` operator are useful to specify the resulting column name.

While all of these operations are quite powerful, sometimes the logic you wish to express is more easily encoded with `if/else` semantics. A simple example of this is encoding the different types of panda as a numeric value⁵. The `when` and `otherwise` functions can be chained together to create the same effect.

Example 3-16. If/Else in Spark SQL

```
/**
 * Encodes pandaType to Integer values instead of String values.
 *
 * @param pandaInfo the input DataFrame
 * @return Returns a DataFrame of pandaId and integer value for pandaType.
 */
def encodePandaType(pandaInfo: DataFrame): DataFrame = {
  pandaInfo.select(pandaInfo("id"),
    (when(pandaInfo("pt") === "giant", 0).
      when(pandaInfo("pt") === "red", 1).
      otherwise(2)).as("encodedType")
  )
}
```

⁵ `StringIndexer` in the ML pipeline is designed for string index encoding.

Specialized DataFrame Transformations for Missing & Noisy Data

Spark SQL also provides special tools for handling missing, null, and invalid data. By using `isNaN` or `isNull` along with filters, you can create conditions for the rows you want to keep. For example, if you have a number of different columns, perhaps with different levels of precision (some of which may be null), you can use `coalesce(c1, c2, ..)` to return the first non-null column. Similarly, for numeric data, `nanvl` returns the first non-NaN value (e.g. `nanvl(0/0, sqrt(-2), 3)` results in 3). To simplify working with missing data, the `na` function on `DataFrame` gives us access to some common routines for handling missing data in [DataFrameNaFunctions](#).

Beyond Row-by-Row Transformations

Sometimes applying a row-by-row decision, as you can with `filter`, isn't enough. Spark SQL also allows us to select the unique rows by calling `dropDuplicates`, but as with the similar operation on RDDs (`distinct`), this can require a shuffle, so is often much slower than `filter`. Unlike with RDDs, `dropDuplicates` can optionally drop rows based on only a subset of the columns, such as an `id` field.

Example 3-17. Drop duplicate panda ids

```
pandas.dropDuplicates(List("id"))
```

This leads nicely into our next section on aggregates and `groupBy` since often the most expensive component of each is the shuffle.

Aggregates and groupBy

Spark SQL has many powerful aggregates, and thanks to its optimizer it can be easy to combine many aggregates into one single action/query. Like with Pandas' DataFrames, `groupBy` returns a special `GroupedData` object on which we can ask for certain aggregations to be performed. Aggregations on Datasets behave differently returning a `GroupedDataset`, as discussed in [“Grouped Operations on Datasets” on page 72](#). `min`, `max`, `avg`, and `sum` are all implemented as convenience functions directly on `GroupedData`, and more can be specified by providing the expressions to `agg`. Once you specify the aggregates you want to compute, you can get the results back as a `DataFrame`.



If you're used to RDDs you might be concerned by `groupBy`, but it is now a safe operation on DataFrames thanks to the Spark SQL optimizer, which automatically pipelines our reductions avoiding giant shuffles and mega records.

Example 3-18. Compute the max panda size by zip code

```
def maxPandaSizePerZip(pandas: DataFrame): DataFrame = {  
    pandas.groupBy(pandas("zip")).max("pandaSize")  
}
```

While this computes the max on a per-key basis, these aggregates can also be applied over the entire Dataframe or all numeric columns in a Dataframe. This is often useful when trying to collect some summary statistics for the data with which you are working. In fact there is a built-in `describe` transformation which does just that, although it can also be limited to certain columns.

Example 3-19. Compute some common summary stats, including count, mean, stddev, and more, on the entire DataFrame

```
pandas.describe()
```



The behaviour of `groupBy` has changed between Spark versions. Prior to Spark 1.3 the values of the grouping columns are discarded by default, while post 1.3 they are retained. The configuration parameter, `spark.sql.retainGroupColumns`, can be set to `false` to force the earlier functionality.

For computing multiple different aggregations, or more complex aggregations, you should use the `agg` api on the `GroupedData` instead directly calling `count`, `mean`, or similar convenience functions. For the `agg` API, you either supply a list of aggregate expressions, a string representing the aggregates, or a map of column names to aggregate function name. Once we've called `agg` with the requested aggregates, we get back a regular `DataFrame` with the aggregated results. As with regular functions, they are listed in [org.apache.spark.sql.functions scaladoc](#). [Table 3-8](#) lists some common and useful aggregates. For our example results in these tables we will consider a `DataFrame` with the schema of `name` field (as a string) and `age` (as an integer), both nullable with values (`{"ikea", null}`, `{"tube", 6}`, `{"real", 30}`).

Example 3-20. Example aggregates using the `agg` API

```
def minMeanSizePerZip(pandas: DataFrame): DataFrame = {  
    // Compute the min and mean  
    pandas.groupBy(pandas("zip")).agg(min(pandas("pandaSize")), mean(pandas("pandaSize")))  
}
```



Computing multiple aggregates with Spark SQL can be much simpler than doing the same tasks with the RDD API.

Table 3-8. Spark SQL aggregate functions for use with *agg* API

Function name	Purpose	Storage requirement	Input Types	Example usage	Example Result
approxCount Distinct	Count approximate distinct values in column ^a	Configurable through <code>rsd</code> (which controls error rate)	All	<code>df.agg(approxCountDistinct(df("age"), 0.001))</code>	2
avg	Average	Constant	Numeric	<code>df.agg(avg(df("age")))</code>	{18}
count	Count number of items (excluding nulls). Special case of <code>"*"</code> counts number of rows	Constant	All	<code>df.agg(count(df("age")))</code>	2
countDistinct	Count distinct values in column	O(distinct elems)	All	<code>df.agg(countDistinct(df("age")))</code>	2
first	Return the first element ^b	Constant	All	<code>df.agg(first(df("age")))</code>	6
last	Return the last element	Constant	All	<code>df.agg(last(df("age")))</code>	30
stddev	Sample standard deviation ^c	Constant	Numeric	<code>df.agg(stddev(df("age")))</code>	16.97...
stddev_pop	Population standard deviation ^c	Constant	Numeric	<code>df.agg(stddev_pop(df("age")))</code>	12.0
sum	Sum of the values	Constant	Numeric	<code>df.agg(sum(df("age")))</code>	36

Function name	Purpose	Storage requirement	Input Types	Example usage	Example Result
sumDistinct	Sum of the distinct values	Constant	Numeric	<code>df.agg(sumDistinct(df("age")))</code>	36
min	Select the minimum value	Constant	Sortable data	<code>df.agg(min(df("age")))</code>	{5}
max	Select the maximum value	Constant	Sortable data	<code>df.agg(max(df("age")))</code>	{30}
mean	Select the maximum value	Constant	Sortable data	<code>df.agg(max(df("age")))</code>	{30}

^a Implemented with HyperLogLog: <https://en.wikipedia.org/wiki/HyperLogLog>.

^b This was commonly used in early versions of Spark SQL where the grouping column was not preserved.

^c Added in Spark 1.6.



In addition aggregates on `groupBy`, you can run the same aggregations on multi-dimensional cubes with `cube` and `rollups` with `rollup`.

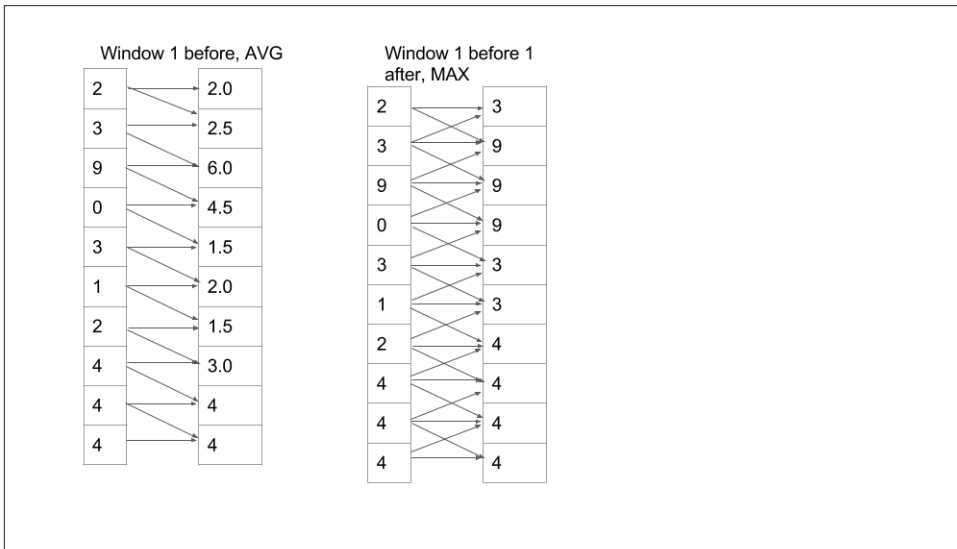
If the built-in aggregation functions don't meet your needs, you can extend Spark SQL using UDFs as discussed in “[Extending with User Defined Functions & Aggregate Functions \(UDFs, UDAFs\)](#)” on [page 72](#), although things can be more complicated for aggregate functions.

Windowing

Spark SQL 1.4.0 introduced windowing functions to allow us to more easily work with ranges or windows of rows. When creating a window you specify what columns the window is over, the order of the rows within each partition/group, and the size of the window (e.g. K rows before and J rows after OR range between values). Using this specification each input row is related to some set of rows, called a frame, that is used to compute the resulting aggregate. Window functions can be very useful for things like computing average speed with noisy data, relative sales, and more.

Example 3-21. Define a window on the +/-10 closest (by age) pandas in the same zip code

```
val windowSpec = Window
  .orderBy(pandas("age"))
  .partitionBy(pandas("zip"))
  .rowsBetween(start = -10, end = 10) // use rangeBetween for range instead
```



Once you've defined a window specification you can compute a function over it. Spark's existing aggregate functions, covered in "Aggregates and groupBy" on page 50, can be computed an aggregate for the window. Window operations are very useful for things like Kalman filtering or many types of relative analysis.

Example 3-22. Compute difference from the average using the window of +/-10 closest (by age) pandas in the same zip code

```
val pandaRelativeSizeFunc = pandas("pandaSize") -
  avg(pandas("pandaSize")).over(windowSpec)

pandas.select(pandas("name"), pandas("zip"), pandas("pandaSize"), pandas("age"),
  pandaRelativeSizeFunc.as("panda_relative_size"))
```



As of this writing, Windowing functions require a HiveContext.

Sorting

Sorting supports multiple columns in ascending or descending order, with ascending as the default. Spark SQL has some extra benefits for sorting as some serialized data can be compared without deserialization.

Example 3-23. Sort by panda age and size in opposite orders

```
pandas.orderBy(pandas("pandaSize").asc, pandas("age").desc)
```

When limiting results, sorting is often used to only bring back the top or bottom K results. When limiting you specify the number of rows with `limit(numRows)` to restrict the number of rows in the DataFrame. Limits are also sometimes used for debugging without sorting to bring back a small result. If, instead of limiting the number of rows based on a sort order, you want to sample your data, ??? covers techniques for Spark SQL sampling as well.

Multi DataFrame Transformations

Beyond single DataFrame transformations you can perform operations that depend on multiple DataFrames. The ones that first pop into our heads are most likely the different types of joins, which are covered in [Chapter 4](#), but beyond that you can also perform a number of set-like operations between DataFrames.

Set Like Operations

The DataFrame set-like operations allow us to perform many operations that are most commonly thought of as set operations. These operations behave a bit differently than traditional set operations since we don't have the restriction of unique elements. While you are likely already familiar with the results of set-like operations from regular Spark and Learning Spark, it's important to review the cost of these operations in [Table 3-9](#).

Table 3-9. Set operations

Operation Name	Cost
<code>unionAll</code>	Low
<code>intersect</code>	Expensive
<code>except</code>	Expensive
<code>distinct</code>	Expensive

Plain Old SQL Queries and Interacting with Hive Data

Sometimes, it's to use regular SQL queries instead of building up our operations on DataFrames. If you are connected to a Hive Metastore we can directly write SQL queries against the Hive tables and get the results as a DataFrame. If you have a DataFrame you want to write SQL queries against, you can register it as a temporary table (or save it as a managed table if you intend to re-use it between jobs). Datasets can also be converted back to DataFrames and registered for querying against.

Example 3-24. Registering/saving tables

```
def registerTable(df: DataFrame): Unit = {  
    df.registerTempTable("pandas")  
    df.saveAsTable("perm_pandas")  
}
```

Querying tables is the same, regardless of whether it is a temporary table, existing Hive table, or newly-saved Spark table.

Example 3-25. Querying a table (permanent or temporary)

```
def querySQL(): DataFrame = {  
    sqlContext.sql("SELECT * FROM pandas WHERE size > 0")  
}
```

In addition to registering tables you can also write queries directly against a specific file path.

Example 3-26. Querying a raw file

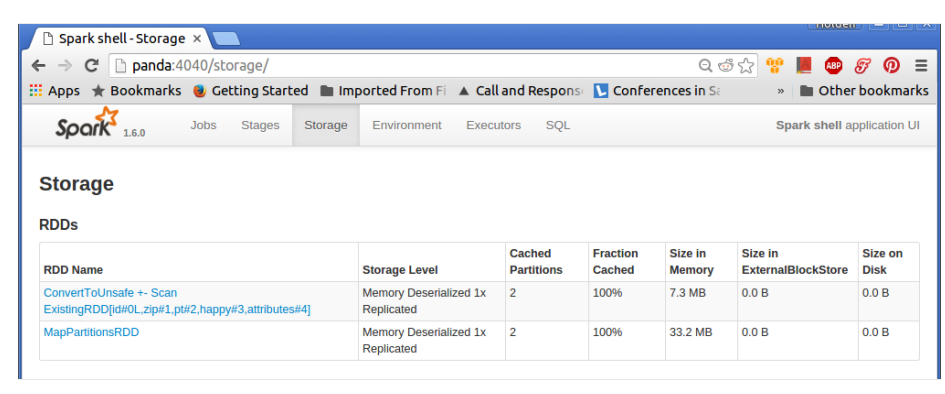
```
def queryRawFile(): DataFrame = {  
    sqlContext.sql("SELECT * FROM parquet.`path_to_parquet_file`")  
}
```

Data Representation in DataFrames & Datasets

DataFrames are more than RDDs of Row objects; DataFrames and Datasets have a specialized representation and columnar cache format. The specialized representation is not only more space efficient, but also can be much faster to encode than even Kryo serialization. To be clear, like RDDs, DataFrames and Datasets are generally lazily evaluated and build up a lineage of their dependencies (except in DataFrames this is called a logical plan and contains more information).

Tungsten

Tungsten is a new Spark SQL component that provides more efficient Spark operations by working directly at the byte level. Looking back on, [Figure 3-1](#), we can take a closer look at the space differences between the RDDs and DataFrames when cached in [Figure 3-3](#). Tungsten includes specialized in-memory data structures tuned for the type of operations required by Spark, improved code generation, and a specialized wire protocol.



The screenshot shows the Spark shell application UI with the 'Storage' tab selected. The table below represents the data shown in the 'RDDs' section of the UI.

RDD Name	Storage Level	Cached Partitions	Fraction Cached	Size in Memory	Size in ExternalBlockStore	Size on Disk
ConvertToUnsafe +- Scan ExistingRDD[jd#0L.zip#1.pt#2.happy#3.attributes#4]	Memory Deserialized 1x Replicated	2	100%	7.3 MB	0.0 B	0.0 B
MapPartitionsRDD	Memory Deserialized 1x Replicated	2	100%	33.2 MB	0.0 B	0.0 B

Figure 3-3. RDD versus Dataframe storage space for same data



For those coming from Hadoop, you can think of Tungsten data types as being `WritableComparable` types on steroids.

Tungsten's representation is substantially smaller than objects serialized using Java or even Kryo serializers. As Tungsten does not depend on Java objects, both on-heap and off-heap allocations are supported. Not only is the format more compact, serialization times can be substantially faster than with native serialization.



Since Tungsten no longer depends on working with Java objects, you can use either on-heap (in the JVM) or off-heap storage. If you use off-heap storage, it is important to leave enough room in your containers for the off-heap allocations - which you can get an approximate idea for from the web ui.

Tungsten's data structures are also created closely in mind with the kind of processing for which they are used. The classic example of this is with sorting, a common and

expensive operation. The on-wire representation is implemented so that sorting can be done without having to deserialize the data again.



In the future Tungsten may make it more feasible to use certain non-JVM libraries. For many simple operations the cost of using BLAS, or similar linear algebra packages, from the JVM is dominated by the cost of copying the data off-heap.

By avoiding the memory and GC overhead of regular Java objects, Tungsten is able to process larger data sets than the same hand-written aggregations. Tungsten became the default in Spark 1.5 and can be enabled in earlier versions by setting `spark.sql.tungsten.enabled` to `true` (or disabled in later versions by setting this to `false`). Even without Tungsten, Spark SQL uses a columnar storage format with Kryo serialization to minimize storage cost.

Data Loading and Saving Functions

Spark SQL has a different way of loading and saving data than core Spark. So as to be able to push down certain types of operations to the storage layer, Spark SQL has its own **Data Source API**. Data Sources are able to specify and control which type of operations should be pushed down to the data source. As developers, you don't need to worry too much about the internal activity going on here, unless the data sources you are looking for are not supported.



Data loading in Spark SQL is not quite as lazy as in regular Spark, but is still generally lazy. You can verify this by quickly trying to load from a data source that doesn't exist.

DataFrameWriter and DataFrameReader

The **DataFrameWriter** and the **DataFrameReader** cover writing and reading from external data sources. The **DataFrameWriter** is accessed by calling `write` on a **DataFrame** or **Dataset**. The **DataFrameReader** can be accessed through `read` on a **SQLContext**.



Spark SQL updated the load/save API in Spark 1.4, so you may see code still using the old-style API without the **DataFrame** reader or writer classes, but under the hood it is implemented as a wrapper around the new API.